Fleet Simulation Engine

Design Document

Project 2 – Abhiram Bhupatiraju

1 Architecture and State

The engine reads JSON inputs from stdin and returns one JSON success/error response per command. The code primarily stays in 5 files: `main.rs` (The Command Line Interface, the place where you type into the terminal), `protocol.rs` (JSON, text format for sending data), `model.rs` (Data, keeps track of the ships ports and cargo. Also defines what those look like), `engine.rs` (Logic for whether ships can move or what would happen on a tick), and `error.rs` (Handles errors.).

All Vessels can be in only 3 types of states: `Idle`, `Transit`, or `Docked`. All eligibility checks decide whether a command is allowed or not.

The engine keeps two copies of the world **Committed State** means the world as it is currently is saved or the locked-in version. **Planned State** is basically a draft of what the world will look like after the next tick. `get_state` returns the planned state (the draft), not the committed state.

Each command is always checked against the committed state, but their effects go into planned state. For example, if an idle vessel receives `dock` and then `load` in the same tick, `load` is rejected because the vessel is not yet docked in committed state. It can load after the next tick.

This was done to prevent commands within one tick from changing each other's eligibility.

Also, immediate mutation was rejected because it would allow the order of each command to affect its validity.

2 Base Simulation

`init` This sets ups the world: ports, travel times between them, and also the ships. `set_destination` requires the vessel to be `Idle` in committed state. A request afterwards will change the previous destination. Travel to the current location is rejected to avoid a trip that has zero distance and is considered unnecessary. A vessel becomes idle **on its arrival tick** and can receive a new command that same tick.

Tick order: (1) Check the entire list. If any item is wrong then then cancel the tick. (2) Take the planned changes and commit them to official. (3) Increment the clock by 1. (4) Ships that arrive become idle at their new port. (5) Add the new cargo to the docks. (6) Next, clear the queued cargo work. (7) Save the world as a snapshot.

3 Follow-on 1: Cargo and Docks

In this follow on we add cargo to the world. Also, ports now are able to have dock inventories which is basically just cargo waiting to be picked up and vessles that can carry the cargo. It also adds new commandsL `dock`, `undock`, `load`, `unload`, and `swap`. Delivery is not automatic: cargo is delivered only when an `unload` command is sent while the vessel is docked at that cargo's destination. It can only do work if it is **docked in committed state**, meaning if it was docked previously before the specific tick happened.

The specification requires `dock_throughput`, but in this implementation, it is optional to ensure that it still is compatible with base commands. If `dock_throughput` is not given, cargo commands are rejected and cargo fields are left out of the output.

Docking edge cases: Sending `dock` or `undock` in the same tick should do nothing the second time. `dock` when already committed docked, or `undock` when it is not moving and is idle should be an error. `undock` after a pending

`dock`, or `dock` after a pending `undock`, is an error, because eligibility always reads committed state. Also, the elgibility always should read the committed state, so any command prior has not taken effect.

Cargo commands should also still be allowed even if it is before or after a pending `undock`. The vessel should still be docked in committed state and the cargo work must be committed before the vessel leaves. **Throughput:** The specification counts each command as one operation. This implementation instead counts the minimum number of operations needed for the net cargo change:

$$C_{\text{dock}} = \sum_{\text{vessels}} \max(L_{\text{net}}, U_{\text{net}})$$

where total loads and unloads are calculated per vessel. Therefore, a load followed by another unload from the same cargo should be **0 operations**; on the other hand, a load and unload of different cargo can combine into one swap at the end. This makes equivalent plans cost the same.

The other implementation decisions I chose to make were: swaps with the same load or unload destinations should always be rejected. The manifest cargo starting at its destination should also be rejected. The `docks` output is an assumption because the provided specification did not define it enough so it was left to speculation.

4 Follow-on 2: History

`get_state_at` returns immutable committed snapshots for ticks from 0 through the current tick. Negative/future ticks are errors; intermediate planned states are not stored.

Full snapshots provide immediate historical lookup but consume increasing memory each tick. Replaying a command log was rejected because it would reduce memory use at the cost of slower queries and greater complexity.

5 Validation, Tests, and Limitations

Invalid commands never mutate state. Validation covers JSON structure, matrix dimensions, vessel status, locations, cargo availability, throughput, and history ranges. The CLI handles malformed JSON, invalid UTF-8, blank lines, CRLF, BOM, and missing final newlines.

Tests cover base/cargo/history examples, command-order edge cases, atomic manifest rejection, and fixed-seed randomized invariants: determinism, cargo conservation, throughput, arrivals, immutable history, and unchanged state after errors.

Known limitations: full snapshots increase memory usage, and state responses return the entire world rather than incremental updates.

6 Build and Run

```
cargo build --release
cargo test --release
make test
make examples
make lint
cargo run --release < examples/base.jsonl
```